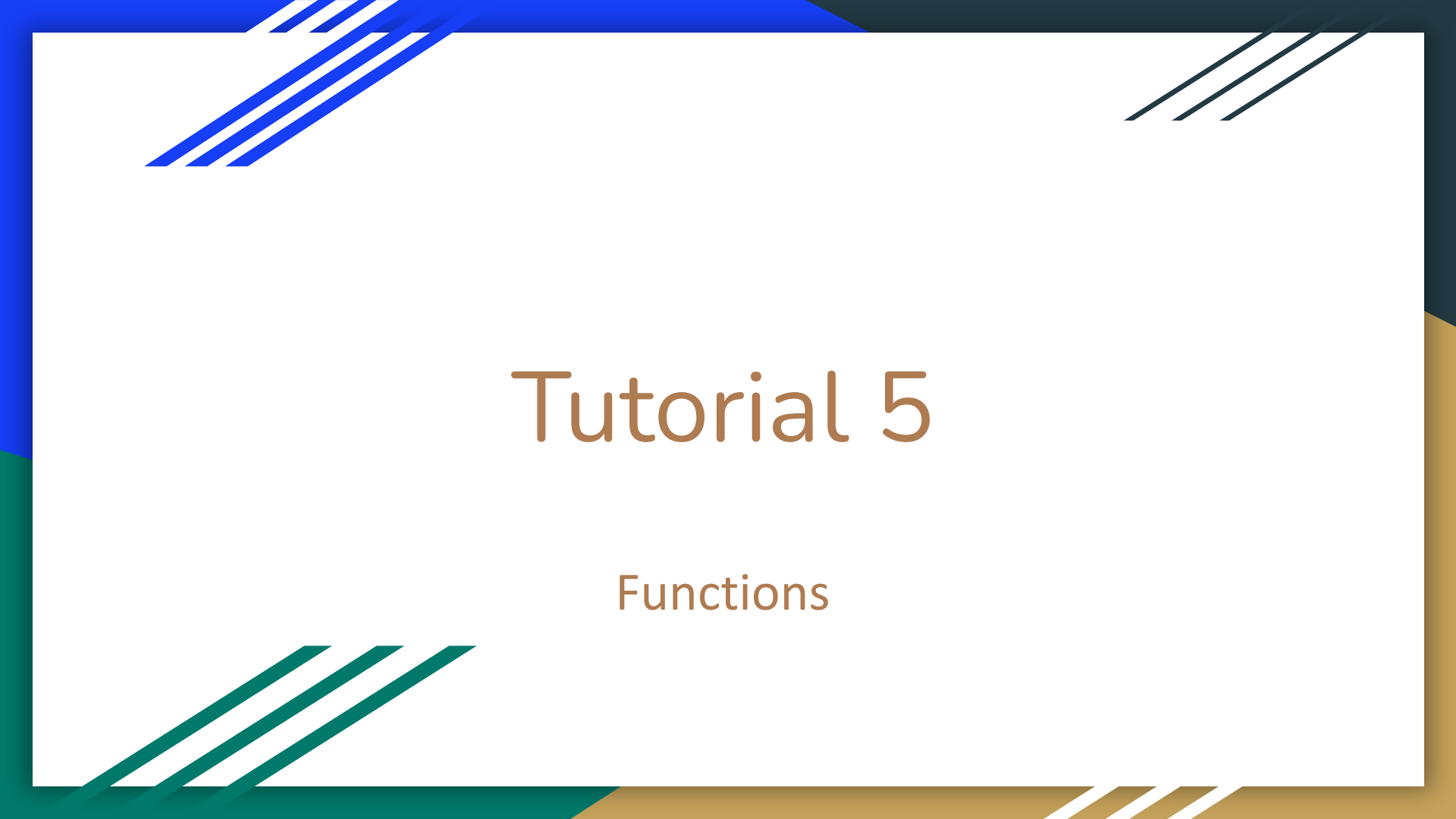




Tutorial 5

Functions

Overview

Revision

- C++ compiler: installation and example use
- Makefiles: syntax, execution order and example use

Functions

- Introduction: main function
- Function definition
- Function call
- Separation of concern: compiling and Linking

Revision: C++ compiler

Code:

```
#include <iostream>
using namespace std;
int main(){
    Cout << "Hello World!" << endl;
}
```

Copy code to main.cpp file

Commands:

- Compile
 - `g++ -c -std=c++98 main.cpp`
 - `g++ -std=c++98 main.o -o main`
- Execute
 - `./main`
- Clean
 - `rm *.o main`

Revision: Makefiles

```
main: main.o
    g++ -std=c++98 main.o -o main

main.o: main.cpp
    g++ -c -std=c++98 main.cpp

run: main
    ./main

clean: main.o main
    rm *.o main
```

Make commands:

- Compile
 - make
- Execute
 - make run
- Clean
 - make clean

Functions:



Functions: Introduction

Definition

- a function is a block of code that performs a specific task.
- It has a name, and it can be called from other parts of the code to execute the task when needed.
- Functions can also take parameters and return values

Main function

- Serves as the entry point for the program.
 - Eg. a house can have multiple doors but there's always the main entrance
- When you run a C++ program, the computer starts executing code from the beginning of the main function

Functions: Syntax

Function definition

```
<ReturnType> <functionName> ( [ Parameter List ] ) {  
    <FunctionBody>  
    [Return Statement]  
}
```

Function call

```
<functionName> ( [ Actual Parameter List ] );  
or  
<type> <variableName> = <functionName> ( [ Actual Parameter List ]
```

Syntax

- Function name:
 - function identifier, has to be unique
- Parameters:
 - expected data to be passed to the function. Has a type and name
- Return type:
 - void, int, float, char, string ... etc
- Return statement:
 - None if return type is void
 - Uses the **return** keyword

Functions: Definition and Call

Definition:

```
void calculateGraph(int x1, int y1, int x2, int y3){  
    //calculate gradient: m  
    //calculate y-intercept: c  
    cout << "y= " << m << "x +" << c << endl;  
}  
  
float gradient(int x1, int y1, int x2, int y3){  
    float m;  
    //calculate gradient: m  
    return m;  
}
```

Function Call:

```
calculateGraph(3,4,5,6);  
// Note: parameter order
```

```
float grdnt = gradient(3,4,5,6);
```


Functions: Examples

Function names

- **functionName.cpp**
- Same/Different function name

Parameter list

- **parameterList.cpp**
- Function parameters vs function call

Return type and return statement

- **returnFunctions.cpp**
- Return type vs return value
- Return type vs function call

Functions: Separation of concern

All functions must be defined before they can be used.

There are **three options**:

- Before the main function in the same file as the main function.
- In the same file as the main function with a forward declaration.
- In a file separate from the main function.
 - Include
 - Compiling and Linking

Functions: Multiple files

// helper.cpp file

```
#include <iostream>
void printSomething()
{
    cout << "Helper \n";
}
```

// helper.h file

```
void printSomething();
```

// main.cpp file

```
#include "helper.h"
#include <iostream>
int main()
{
    cout << "Main Function\n";
    printSomething();
}
```

Why Include .h Files and Not .cpp Files in other files?

Header Files (.h):

- They announce what functions and objects a program has.
- Prevents duplication – They stop the same code from being included multiple times, which can confuse the compiler. (remember to add the #ifndef)
- Save time – They only declare things, they don't define them. If the .cpp is changed, the other files do not need to be changed since they only use the .h.
- Organize – They keep the code clean and easy to understand.

Source Files (.cpp)

- They define how the functions actually work.
- If included everywhere, they would create multiple functions all doing the same thing – won't work because the program wouldn't know which one to use. (multiple definition errors)

Functions: Lecture 14 example

- Problem
 - calculate the **perimeter** and **area** of **Squares, Rectangles, Circles** and **Ellipses**.
- Break down
 - 2 x 4 helper functions
- Solutions
 - functions defined before the main function
 - using forward declarations
 - Splitting the functions into .h and .cpp files